

Trabajo Práctico 1

Desarrollo de la propuesta por escrito en un proyecto

1.1 Equipo y roles

Integrantes:

- ❖ Fistarol Mia
- ❖ Barbero Gianella

Decidimos optar por roles rotativos para garantizar la participación equitativa en todas las etapas del proyecto.

Roles:

- ❖ Desarrollo Back-End: lógica del sistema, rutas y conexión a base de datos.
- ❖ Desarrollo Front-End: interfaz e interacción con el usuario.
- ❖ Diseño de Base de Datos: estructura y relaciones de la información.
- ❖ Testing y documentación: pruebas, validaciones y registro del desarrollo.

1.2 Descripción del proyecto y justificación

Decidimos ampliar el vivero original agregando características ficticias para poder trabajar con un sistema más completo. La idea es simular un caso más real, incorporando funcionalidades complejas, lo que nos permite practicar mejor la lógica del back-end. Al ser un proyecto ficticio, también tenemos más libertad para diseñar y agregar funcionalidades nuevas sin limitarnos a un caso real.

Además, pensamos en seguir incluyendo la información de la fundación pero el apartado del vivero separarnos un poco de la estética infantil y acercarnos a algo más moderno y adulto.

El proyecto consiste en el desarrollo de un sistema web para un vivero ampliado, el sistema permitirá:

- ❖ Gestionar un catálogo ampliado de plantas y macetas.
- ❖ Aplicar filtros de búsqueda según diferentes características (tipo, precio, tipo de ambiente, etc.).
- ❖ Ver cuidados de plantas.

- ❖ Administrar usuarios y registrar pedidos realizados.

En resumen, el sistema busca mejorar la organización de la información y brindar una experiencia más dinámica al usuario, permitiendo la exploración de productos y la simulación de compras.

1.3 Cronograma de trabajo

Semana	Fecha	Actividad
Semana 1	23/03 al 27/03	Planteamiento inicial de la elección del proyecto
Semana 2	30/03 al 03/04	Definir alcance y funcionalidades
Semana 3 Entrega TP1	06/04 al 10/04	Reestructuración del proyecto anterior, definir objetivos y cronograma, herramientas de gestión y GitHub
Semana 4	13/04 al 17/04	Web funcional (lógica y acciones)
Semana 5	20/04 al 24/04	Poner en funcionamiento con MockAPI
Semana 6 Entrega TP2	27/04 al 01/05	Modularizar, completar documentación y entrega final
Semana 7	04/05 al 08/05	Ajustes post-entrega Implementar Modelo Vista Controlador
Semana 8	11/05 al 15/05	Creación de APIs Desarrollo de operaciones CRUD completas
Semana 9	18/05 al 22/05	Front-end administración
Semana 10 Entrega TP3	25/05 al 29/05	Creación de base de datos Informe completo

Semana 11	01/06 al 05/06	Implementación de login y manejo de usuarios
Semana 12	08/06 al 12/06	Seguridad del sistema y validaciones (JWT y cookies) Implementar variables de entorno
Semana 13 Entrega Final	15/06 al 19/06	Testing final, correcciones y entrega final

Herramienta de organización

Usaremos **Trello** para la gestión de tareas, organización de actividades y seguimiento del progreso del proyecto.

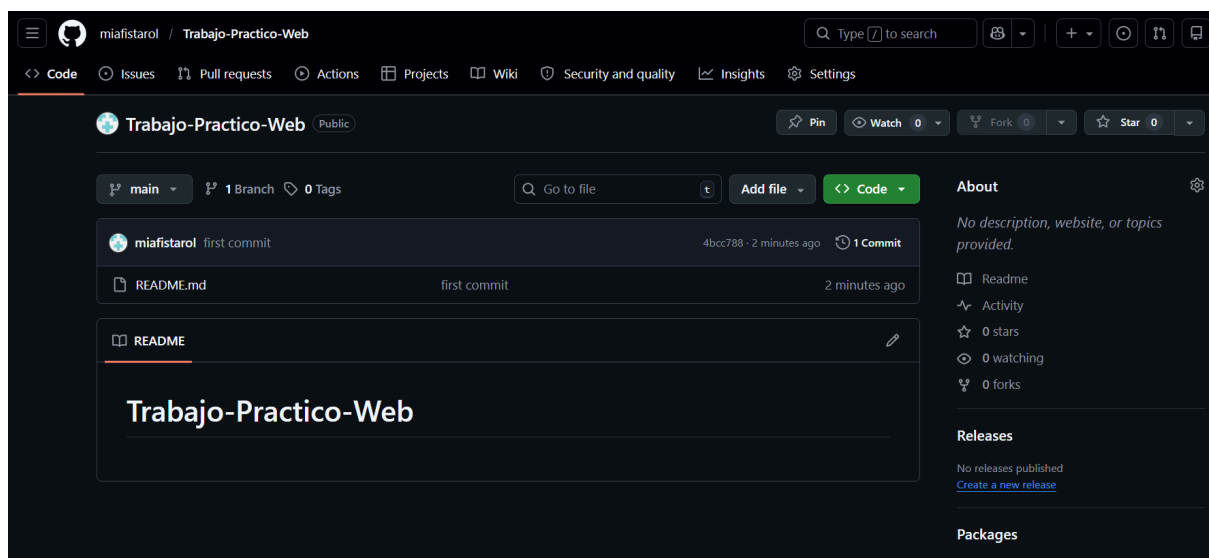
<https://trello.com/invite/b/69d8113bda3738d8385049b9/ATTI4b7c735d3430d43600550c150e0b986c94D792C7/trabajo-practico-web>

1.3 Control de versiones y seguimiento

Se utilizará:

- ❖ Git para el control de versiones.
- ❖ GitHub como repositorio del proyecto.

Podremos registrar los cambios realizados durante el desarrollo, mantener un historial del proyecto, mantenernos actualizadas de cambios hechos por cada una y facilitarnos el trabajo en equipo.



Trabajo Práctico 2

Back-end para servir el sitio web

2.1 Servidor Express Backend

En este trabajo práctico se desarrolló un backend utilizando Node.js y Express con el objetivo de servir a la aplicación web. El sistema implementado permite obtener datos desde una API externa (MockAPI) y servirlos al frontend mediante endpoints propios, logrando así una arquitectura cliente-servidor funcional.

El sistema se basa en una arquitectura de tres capas:

Frontend → Backend → API externa (MockAPI)

- ❖ El frontend realiza solicitudes HTTP mediante fetch
- ❖ El backend actúa como intermediario procesando las solicitudes
- ❖ La API externa provee los datos en formato JSON

2.1 Tecnologías que usamos

- ❖ Node.js
- ❖ Express.js
- ❖ MockAPI
- ❖ JavaScript (ES Modules)
- ❖ HTML, CSS y JavaScript (frontend)

2.3 Desarrollo

Se creó un servidor utilizando Express que escucha en el puerto 3000.

Se implementó un endpoint:

GET /productos

Este endpoint realiza una solicitud a la API externa y devuelve los datos al cliente. Se utilizó MockAPI como fuente de datos.

Los datos fueron cargados previamente desde un archivo JSON local mediante un script.

URL de la **API utilizada**:

<https://69f3c943bd2396bf53105d60.mockapi.io/productos>

Middleware

Se implementó el middleware cors para permitir la comunicación entre el frontend y el backend, ya que se ejecutan en diferentes puertos.

2.4 Problemas encontrados y soluciones

Durante el desarrollo se presentaron los siguientes problemas:

- ❖ Error CORS: se solucionó utilizando el middleware cors.
- ❖ Error de rutas en MockAPI: se corrigió la URL incluyendo /api/.
- ❖ Errores en formato de datos: se adaptó el consumo de la API para manejar correctamente el JSON recibido.

Trabajo Práctico 3

Creación del CRUD para administrar Front-End

3. Avances

Durante esta etapa del trabajo práctico se avanzó en la reorganización y estructuración del sistema utilizando una arquitectura basada en el patrón Modelo Vista Controlador (MVC). El objetivo principal fue modularizar el código para mejorar su mantenimiento, escalabilidad y comprensión.

3.1 Implementación de arquitectura MVC

Se separó la lógica del sistema en distintos módulos:

- Modelo: encargado de la interacción con la base de datos PostgreSQL y de ejecutar consultas SQL.
- Controlador: responsable de recibir las solicitudes, procesar la lógica necesaria y coordinar la comunicación entre el modelo y la vista.
- Vista: utilizada para definir la respuesta enviada al cliente, principalmente en formato JSON.
- Rutas: encargadas de definir los endpoints de la API y vincularlos con sus respectivos controladores.

3.2 Integración con PostgreSQL

Se avanzó en la conexión entre el backend desarrollado en Node.js y la base de datos PostgreSQL. Para ello se realizaron pruebas de consultas y obtención de datos desde la aplicación.

3.3 Implementación de Docker

Se incorporó Docker para contenerizar la base de datos PostgreSQL y facilitar la configuración del entorno de desarrollo. Esto permite ejecutar el sistema de manera más consistente y portable entre distintos equipos.

3.4 Herramientas nuevas

- ❖ PostgreSQL
- ❖ Docker

4. Conceptos y funcionalidades explicadas

4.1 Uso de parámetros en consultas SQL

Para la construcción de las consultas SQL que reciben datos externos, como un id o los campos de un producto, se utilizaron parámetros posicionales en lugar de concatenar los valores directamente en el string. La sintaxis es \$1, \$2, \$3, y así sucesivamente, donde cada número representa la posición del valor en el array que se pasa como segundo argumento a `pool.query()`.

Esta práctica evita la inyección SQL, que es un tipo de ataque donde un usuario malintencionado puede manipular la consulta enviando valores especiales como input. Al usar parámetros, la librería pg se encarga de sanitizar los valores antes de ejecutar la consulta.

4.2 RETURNING * en operaciones de escritura

En las operaciones de inserción, modificación y eliminación se agregó la cláusula `RETURNING *` al final de la consulta SQL. Esto le indica a PostgreSQL que, luego de ejecutar la operación, devuelva la fila afectada completa como si fuera un `SELECT`. Esto es útil porque permite responder al cliente con el objeto actualizado sin necesidad de hacer una segunda

consulta a la base de datos para obtenerlo. En el caso del INSERT, además devuelve el id generado automáticamente por el SERIAL de PostgreSQL.

4.3 Input oculto para distinguir alta y edición

```
<input type="hidden" id="input-id">
```

El formulario del panel de administración se utiliza tanto para crear un producto nuevo como para editar uno existente. Para distinguir entre ambos casos se incorporó un campo `<input type="hidden">` en el HTML, invisible para el usuario pero accesible desde JavaScript.

Cuando el usuario clickea el botón Editar de una fila, la función `prepararEdicion` hace un GET a la API, carga los datos del producto en el formulario y guarda el id en ese campo oculto. Al hacer submit, el código lee ese valor: si está vacío realiza un POST para crear, si tiene un valor realiza un PUT para modificar.

4.4 Configuración de variables de entorno con dotenv y habilitación de CORS

Las credenciales de la base de datos, como el usuario, la contraseña y el nombre de la base, se almacenaron en un archivo `.env` en la raíz del proyecto. Este archivo no se sube al repositorio y permite cambiar la configuración sin modificar el código fuente.

`dotenv.config()` lee el archivo `.env` y carga todas las variables en `process.env`, que es el objeto global de Node.js donde viven las variables de entorno del sistema. A partir de esa línea se puede acceder a los valores con `process.env.DB_PASSWORD`, por ejemplo.

Por otro lado, se incorporó el middleware `cors` para habilitar el intercambio de recursos entre distintos orígenes. Sin esta configuración, el navegador bloquea por defecto los requests que provienen de un dominio o puerto diferente al del servidor.

4.5 Formateo de respuestas en la capa Vista

La capa Vista tiene como responsabilidad definir la forma que va a tener el JSON que se le envía al cliente. Recibe el objeto crudo que devuelve PostgreSQL, cuyos campos tienen los nombres exactos de las columnas de la tabla, y construye un objeto nuevo con la estructura deseada para la API.

Por ejemplo, la columna en la base de datos se llama `id_planta` pero en la respuesta de la API se expone como `id`. Centralizar esta transformación en un solo lugar significa que si en el futuro cambia el nombre de una columna en la base de datos, solo hay que actualizar la vista y no buscar el cambio en todo el proyecto. Para el caso de múltiples resultados se utiliza `formatearTodos`, que aplica `formatearUno` a cada elemento del array usando `map`.

4.6 Uso de inteligencia artificial como herramienta de apoyo

Durante el desarrollo de este trabajo práctico se utilizó inteligencia artificial como herramienta de consulta y asistencia. El uso se orientó principalmente a dos aspectos: la generación del panel de administración y el aprendizaje de conceptos nuevos.

En cuanto al panel de administración, se solicitó ayuda para generar el código base del archivo `admin/index.html` y su hoja de estilos `admin.css`. Para su desarrollo se indicó que el diseño debía mantener coherencia visual con el sitio reutilizando la paleta de colores y el estilo general del vivero. El archivo `admin.js` también fue generado con asistencia de la IA y luego revisado y corregido en conjunto, ya que al adaptar el código a la estructura real de la base de datos surgieron errores como variables declaradas dos veces y referencias a campos que ya no existían. Ese proceso de corrección fue parte del aprendizaje.

Además, la IA se utilizó como herramienta explicativa para entender y fijar fragmentos de código tanto nuevos como ya vistos en las clases.

Trabajo Práctico 4

Implementación de autenticación, accesos y seguridad

5.

5.1 Persistencia de datos en base de datos para usuarios registrados

Se creó una tabla usuarios en la base de datos PostgreSQL existente, se decidió no incluir una columna de rol/privilegios porque el sistema solo contempla un único nivel de acceso: administradores del vivero. Esto evita agregar complejidad innecesaria al modelo de datos para un caso de uso que no la requiere.

La restricción UNIQUE sobre la columna usuario impide que existan dos cuentas con el mismo nombre de usuario a nivel de base de datos, como capa adicional de integridad además de la validación que se realiza en el controlador.

Siguiendo la arquitectura MVC ya establecida en TP3, se creó el módulo completo usuarios (modelo, controlador, rutas y vista), replicando el mismo patrón usado en el módulo productos.

5.2 Hashing de claves para mejorar la seguridad de las credenciales

Para evitar almacenar contraseñas en texto plano, se utilizó la librería bcryptjs. El hashing se centralizó en un módulo hash.mjs con dos funciones:

- hashClave(clave): genera el hash a partir de la contraseña en texto plano, antes de guardarla.
- compararClave(clave, hash): compara una contraseña ingresada contra el hash almacenado, sin necesidad de revertir el hash (operación que no es posible, ya que bcrypt es una función de un solo sentido).

Decisión de diseño: se optó por realizar el hashing dentro del modelo de usuarios (modelo.usuario.mjs). De esta forma, el modelo nunca permite que se inserte una contraseña en texto plano en la base de datos, independientemente de desde dónde se lo invoque en el futuro. Esto refuerza la seguridad a nivel de la capa de datos.

5.3 Implementación de autenticación y control de acceso mediante JWT y cookies

El protocolo HTTP no mantiene estado entre peticiones (cada request es independiente). Para que el servidor pueda "recordar" que un usuario ya inició sesión sin tener que volver a pedir las credenciales en cada petición, se implementó autenticación basada en JSON Web Tokens (JWT). Al loguearse correctamente, el servidor firma un token que contiene los datos mínimos necesarios del usuario (id_usuario, usuario) usando una clave secreta que solo el servidor conoce. Ese token tiene una validez de 2 horas y luego deja de ser válido y se requiere un nuevo login.

FLUJOS DE IMPLEMENTACIÓN

- POST /api/v1/usuarios/registro — crea un nuevo usuario administrador (clave hasheada antes de guardar).
- POST /api/v1/usuarios/login — valida usuario y clave contra la base de datos, y si son correctas, firma el JWT y lo entrega en una cookie httpOnly.
- POST /api/v1/usuarios/logout — elimina la cookie del lado del cliente.
- GET /api/v1/usuarios/sesion — endpoint adicional que permite al frontend consultar si existe una sesión activa, sin exponer información sensible. Fue necesario porque, al ser la cookie httpOnly, el JavaScript del cliente no puede determinar por sí mismo si hay sesión iniciada; debe preguntarle al servidor.

Se implementaron dos middlewares distintos en autenticacion.mjs, según el tipo de recurso a proteger:

- verificarSesion: protege endpoints de la API (operaciones POST, PUT y DELETE de productos).
- verificarSesionPagina: protege páginas HTML del panel de administración. Si no hay token válido, redirige (HTTP redirect) a la

página de login, ya que en este caso quien realiza la petición es directamente el navegador esperando una página, no una llamada de API.

Se implementó un componente de interfaz dinámico en la web pública del vivero: el ítem del menú de navegación cambia entre "Iniciar sesión" y "Panel de administración" según exista o no una sesión activa, consultando el endpoint `GET /api/v1/usuarios/sesion` al cargar cada página. Dentro del panel de administración, al estar siempre protegido por el middleware (es decir, sólo es accesible habiendo iniciado sesión), se incorporaron de forma incondicional un enlace para volver a la web pública y un botón de cierre de sesión.

5.4 Habilitación de CORS

En la implementación previa (TP3) se había incorporado el middleware `cors()` sin restricciones, replicando una decisión tomada en TP2 cuando frontend y backend efectivamente corrían en puertos distintos (frontend consumiendo MockAPI).

Al revisar la arquitectura actual del proyecto, se determinó que **CORS no es necesario** y se decidió retirarlo. CORS es una política de seguridad que aplican los navegadores únicamente cuando una página realiza una petición hacia un origen distinto al propio (entendiendo "origen" como la combinación de protocolo, dominio y puerto). En el estado actual del proyecto, tanto la web pública como el panel de administración y la API se sirven desde el mismo servidor Express, en el mismo puerto. Por lo tanto, nunca se produce una petición de origen cruzado, y el navegador no aplica ninguna restricción que sea necesario sortear con `cors()`.

5.5 Uso de variables de entorno

Se amplió el archivo `.env`, ya utilizado desde el TP3 para las credenciales de la base de datos, agregando una nueva variable para la clave secreta de firma de los JWT. Esta clave se utiliza tanto para firmar como para verificar los tokens, y nunca se encuentra escrita directamente en el código fuente. El archivo `.env` continúa excluido del repositorio mediante `.gitignore`, evitando exponer públicamente tanto las credenciales de la base de datos como la clave de firma de los tokens.

5.6 Uso de inteligencia artificial como herramienta de apoyo

Durante el desarrollo de este trabajo práctico se utilizó inteligencia artificial como herramienta de consulta y asistencia. El uso se orientó principalmente a dos aspectos: la generación del panel de administración y el aprendizaje de conceptos nuevos.

Comprender en profundidad los conceptos de autenticación, autorización, JWT y cookies httpOnly antes de implementarlos, discutiendo el porqué de cada decisión (por ejemplo, por qué usar cookies httpOnly en lugar de localStorage, o por qué un mensaje de error genérico en el login).
Recibir una guía paso a paso para la implementación, escribiendo el código siguiendo el mismo estilo y convenciones ya establecidas en los módulos existentes del proyecto (estructura MVC, manejo de errores, nombres de variables).

Diagnosticar y resolver errores concretos surgidos durante las pruebas (el problema de paquetes de bcrypt, la desincronización de la secuencia de PostgreSQL, y el análisis de la relación faltante entre plantas y cuidados).

Evaluar de forma crítica si una práctica mencionada en el material de la materia (en este caso, el uso de CORS) era efectivamente necesaria para la arquitectura real del proyecto, en lugar de incorporar sin verificar su pertinencia.

Trello

<https://trello.com/invite/b/69d8113bda3738d8385049b9/ATTI4b7c735d3430d43600550c150e0b986c94D792C7/trabajo-practico-web>

GitHub NUEVO

<https://github.com/miafistarol/Trabajo-Practico-Web-vivero.git>